

In-Context Learning Theory

gradient descent, implicit algorithms, Bayesian inference, meta-learning

In-context learning(ICL)은 모델 파라미터를 바꾸지 않은 채 prompt 안의 예시만으로 새로운 prediction rule을 구성하는 현상이다. 이번 주의 질문은 “Transformer가 ICL을 할 수 있는가?”가 아니라, 그 forward pass를 어떤 학습 알고리즘으로 해석할 수 있는가이다. 대표적인 설명은 세 갈래다. 첫째, attention이 context example들로부터 gradient-descent update를 계산한다는 **optimization view**; 둘째, hidden activation 안에서 ridge regression 같은 통계적 estimator를 구현한다는 **algorithmic view**; 셋째, pretraining에서 습득한 latent concept에 대해 posterior inference를 수행한다는 **Bayesian view**이다. 이 노트는 세 관점이 서로 경쟁하는 단일 가설이 아니라, 서로 다른 data-generating model과 architecture에서 성립하는 수학적 진술임을 분리해서 정리한다.

Primary readings

Johannes von Oswald et al., **Transformers Learn In-Context by Gradient Descent**, ICML 2023.
Sang Michael Xie et al., **An Explanation of In-context Learning as Implicit Bayesian Inference**, ICLR 2022.

Ekin Akyürek et al., **What Learning Algorithm Is In-Context Learning? Investigations with Linear Models**, ICLR 2023.

Shivam Garg et al., **What Can Transformers Learn In-Context? A Case Study of Simple Function Classes**, NeurIPS 2022.

Contents

1	기호와 문제 설정	2
1.1	논문별 기호 대응표	2
1.2	ICL을 수식으로 정의하면	2
1.3	세 관점의 질문이 서로 다르다	3
2	Linear regression을 공통 실험대로 만들기	4
2.1	왜 모든 논문이 linear regression으로 시작하는가	4
2.2	OLS: context가 충분할 때의 기준점	4
2.3	Ridge regression	5
2.4	Gradient descent가 OLS로 가는 과정	5
2.5	Garg et al.: 함수족 자체를 in-context learn할 수 있는가	6
3	von Oswald et al.: attention이 GD step을 계산할 수 있다	7
3.1	먼저 reference GD update	7
3.2	Linear self-attention	7
3.3	Proposition 1의 핵심 construction을 직접 계산	8
3.4	왜 data를 업데이트하는 것이 parameter를 업데이트하는 것과 같은가	9
3.5	Fast-weight 관점	9
3.6	여러 layer와 curvature correction	10
4	Akyürek et al.: Transformer 안의 암묵적 학습 알고리즘	11
4.1	질문을 더 강하게 바꾸기	11
4.2	GD implementation theorem	11
4.3	Ridge regression을 online update로 쓰기	11
4.4	위 보충 유도의 마지막 식을 다시 확인	12
4.5	왜 depth와 noise에 따라 다른 알고리즘처럼 보일 수 있는가	12
5	Bayesian linear regression: optimization과 inference가 만나는 곳	14
5.1	Gaussian prior와 likelihood	14
5.2	Posterior를 완전히 유도	14
5.3	Posterior predictive	15
5.4	GD, ridge, Bayes가 언제 같은가	16
6	Xie et al.: ICL을 latent-concept Bayesian inference로 보기	17
6.1	Pretraining distribution의 latent concept	17
6.2	Prompt가 왜 posterior를 바꾸는가	17
6.3	Prompt와 pretraining distribution의 mismatch	18
6.4	보충: 단순 IID mixture에서는 posterior concentration이 왜 일어나는가	18
6.5	Bayesian view에서 demonstration order가 중요할 수 있는 이유	19
7	Meta-learning으로 세 관점을 하나의 그림에 놓기	20
7.1	Outer loop와 inner loop	20
7.2	MAML과 ICL의 차이	20
7.3	Amortized inference 관점	21
7.4	세 관점이 동시에 참일 수 있는 예	21

8	무엇이 theorem이고 무엇이 해석인가	22
8.1	네 논문의 claim을 같은 강도로 읽으면 안 된다	22
8.2	Can implement와 does implement	22
8.3	Linear attention과 softmax attention	22
8.4	Toy regression에서 자연어 ICL로 갈 때 추가되는 것	23
8.5	분포 밖 일반화와 task recognition	23
9	발표 준비용 압축 정리	24
9.1	칠판에 반드시 남길 열두 개 식	24
9.2	발표 순서 추천	25
9.3	자주 나오는 질문	25
9.4	수학적 실수 체크리스트	25
9.5	30초 결론	26
	참고문헌 및 원문에서 확인할 위치	27

1

기호와 문제 설정

1.1 논문별 기호 대응표

같은 ICL 문제를 논문마다 다른 관점에서 쓰기 때문에 먼저 기호를 맞춘다. 이 노트에서는 task 를 τ , task-specific parameter 를 w_τ 또는 latent concept θ , model의 고정된 meta-parameter 를 Θ 로 쓴다.

개념	이 노트	von Oswald / Akyürek	Xie et al.
고정 model parameter	Θ	attention/Transformer parameter θ	pretrained LM parameter는 이상적으로 p_{pre} 를 모델링
현재 task	τ	regression teacher W_τ 또는 w	latent concept θ^*
context	$D_n = \{(x_i, y_i)\}_{i=1}^n$	token $e_i = (x_i, y_i)$	prompt examples $O_1, \dots, O_n$
query	x_*	x_{test} , query token	마지막 query observation
ICL predictor	$F_\Theta(D_n, x_*)$	$\hat{y}_\Theta$	posterior-predictive LM output
inner update	activation 변화	GD / ridge algorithm의 암묵적 실행	posterior $p(\theta \mid \text{prompt})$ 변화
outer learning	Θ 학습	여러 regression task에 대한 meta-training	next-token pretraining

1.2 ICL을 수식으로 정의하면

Task distribution $p(\tau)$ 에서 하나의 task를 뽑고, 그 task의 data distribution $p_\tau(x, y)$ 에서 context와 query를 만든다고 하자.

$$\tau \sim p(\tau), \quad D_n = \{(x_i, y_i)\}_{i=1}^n, \quad (x_*, y_*) \sim p_\tau.$$

ICL model은

$$\hat{y}_* = F_\Theta(D_n, x_*)$$

를 출력한다. 가장 중요한 점은 inference 동안

$$\Theta_{\text{before}} = \Theta_{\text{after}}$$

라는 것이다. 즉 optimizer가 model parameter를 업데이트하지 않는다. 그럼에도 D_n 이 바뀌면 prediction rule이 바뀌므로, 계산 내부에서는 어떤 형태의 task adaptation이 일어나야 한다.

Outer training objective는 예를 들어 squared error에서

$$\min_{\Theta} \mathbb{E}_{\tau, D_n, (x_*, y_*)} \left[(F_\Theta(D_n, x_*) - y_*)^2 \right]$$

로 쓸 수 있다.

핵심 ICL에서 “learning”은 두 층위로 분리해야 한다. Training 때는 Θ 가 여러 task에 걸쳐 천천히 바뀐다. Test-time ICL에서는 Θ 는 고정이고 context가 hidden state와 attention computation을 바꾼다. 따라서 “parameter update가 없으니 learning이 아니다”라는 말과 “forward pass 안에서 learning algorithm이 실행된다”는 말은 서로 모순이 아니다.

1.3 세 관점의 질문이 서로 다르다

관점	묻는 질문	대표적인 수학 객체
Function-class	어떤 함수족을 context로 학습할 수 있는가?	risk, OLS, sparse regression, neural net teacher
Optimization	attention layer가 어떤 update rule을 계산하는가?	gradient, fast-weight matrix, curvature correction
Bayesian	context가 어떤 latent task posterior를 만드는가?	prior, likelihood, posterior predictive
Meta-learning	왜 이런 inner algorithm이 training에서 생기는가?	task distribution, outer objective, amortized inference

이 구분을 유지하면 뒤의 논문들이 훨씬 덜 충돌해 보인다.

발표에서 이렇게 말하기 “In-context learning에서 parameter는 inference 중에 바뀌지 않습니다. 대신 context가 forward computation을 바꾸고, 그 computation 자체가 task-specific estimator를 만들어 냅니다. 이번 주는 그 estimator를 gradient descent, ridge regression, Bayesian posterior라는 세 언어로 해석하는 내용입니다.”

2

Linear regression을 공통 실험대로 만들기

2.1 왜 모든 논문이 linear regression으로 시작하는가

가장 단순한 task family를

$$y = w_\tau^\top x + \varepsilon$$

로 둔다. 각 prompt마다 새로운 w_τ 를 뽑는 것이 핵심이다. 예를 들어

$$w_\tau \sim \mathcal{N}(0, I_d), \quad x_i \sim \mathcal{N}(0, I_d), \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

라고 하자.

Context matrix를

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad y = (y_1, \dots, y_n)^\top$$

로 두면

$$y = Xw_\tau + \varepsilon.$$

같은 Transformer parameter Θ 가 prompt마다 다른 w_τ 를 추론해야 하므로, 이 setting은 “weight에 task를 외워 둔 것”과 “context에서 task를 추정하는 것”을 분리하기 좋다.

2.2 OLS: context가 충분할 때의 기준점

Noise가 작고 $X^\top X$ 가 invertible하면 ordinary least squares는

$$\hat{w}_{\text{OLS}} = \arg \min_w \frac{1}{2} \|Xw - y\|^2$$

이고 normal equation은

$$X^\top (X\hat{w} - y) = 0.$$

따라서

$$X^\top X\hat{w} = X^\top y,$$

즉

$$\boxed{\hat{w}_{\text{OLS}} = (X^\top X)^{-1} X^\top y}.$$

Query prediction은

$$\hat{y}_* = x_*^\top \hat{w}_{\text{OLS}}.$$

$n < d$ 라서 underdetermined이면 minimum-norm interpolator는 full row rank 아래

$$\hat{w}_{\min} = X^\top (XX^\top)^{-1} y$$

가 된다.

주의 “Transformer가 OLS와 비슷한 test error를 낸다”와 “Transformer 내부에서 실제로 $(X^\top X)^{-1}$ 을 계산한다”는 같은 진술이 아니다. Behavioral equivalence와 mechanistic equivalence를 구분해야 한다.

2.3 Ridge regression

Regularization을 넣으면

$$\hat{w}_\lambda = \arg \min_w \left\{ \frac{1}{2} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2 \right\}.$$

Gradient를 0으로 두면

$$X^\top(Xw - y) + \lambda w = 0,$$

따라서

$$(X^\top X + \lambda I)w = X^\top y,$$

즉

$$\hat{w}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

Noise가 있거나 context가 짧을 때 ridge가 자연스럽게 등장하는 이유는 뒤의 Bayesian 해석에서 정확히 연결된다.

2.4 Gradient descent가 OLS로 가는 과정

Loss를

$$L(w) = \frac{1}{2n} \|Xw - y\|^2$$

로 두면

$$\nabla L(w) = \frac{1}{n} X^\top (Xw - y).$$

따라서 GD는

$$w_{t+1} = w_t - \eta \frac{1}{n} X^\top (Xw_t - y).$$

$$A = \frac{1}{n} X^\top X, \quad b = \frac{1}{n} X^\top y$$

라고 두면

$$w_{t+1} = (I - \eta A)w_t + \eta b.$$

$w_0 = 0$ 이면 한 단계씩 전개하여

$$w_1 = \eta b,$$

$$w_2 = (I - \eta A)\eta b + \eta b,$$

그리고 일반적으로

$$w_t = \eta \sum_{s=0}^{t-1} (I - \eta A)^s b.$$

A 가 invertible이면 geometric-series identity로

$$\eta \sum_{s=0}^{t-1} (I - \eta A)^s = A^{-1} [I - (I - \eta A)^t].$$

따라서

$$w_t = A^{-1} [I - (I - \eta A)^t] b.$$

모든 eigenvalue에 대해 $|1 - \eta \lambda_i(A)| < 1$ 이면

$$(I - \eta A)^t \rightarrow 0,$$

그래서

$$w_t \rightarrow A^{-1}b = (X^\top X)^{-1}X^\top y = \hat{w}_{\text{OLS}}.$$

보충 유도 — 논문 밖의 독자용 전개 이 계산은 뒤에서 매우 중요하다. “ICL이 GD를 한다”와 “ICL이 OLS와 같은 predictor를 낸다”는 finite depth에서는 다르지만, 충분히 많은 안정적인 GD step을 허용하면 둘은 같은 해로 수렴할 수 있다. 따라서 output만 관찰하면 내부 알고리즘을 식별하기 어려울 수 있다.

2.5 Garg et al.: 함수족 자체를 in-context learn할 수 있는가

Garg et al.은 매 training sequence마다 새로운 target function f 를 뽑고

$$(x_1, f(x_1)), \dots, (x_n, f(x_n)), x_{n+1}$$

을 Transformer에 넣는다. Training loss는 각 prefix에서의 next label prediction error를 평균낸 형태다.

핵심 실험 결과는 다음과 같다.

원 논문 기준 표준 Transformer를 처음부터 meta-train하면 unseen linear function을 context만으로 학습할 수 있고, 충분한 in-context example에서 least-squares predictor와 비슷한 성능을 낸다. 더 나아가 sparse linear function, two-layer neural network, decision tree 같은 더 복잡한 function class에서도 task-specific baseline에 근접하거나 능가하는 결과를 보였다.

이 논문은 mechanism을 증명하는 논문이라기보다, “ICL을 하나의 supervised learning problem으로 엄밀하게 실험할 수 있다”는 공통 testbed를 만든 역할이 크다.

발표에서 이렇게 말하기 “Linear regression을 쓰는 이유는 쉽기 때문만이 아닙니다. 같은 task family에서 OLS, ridge, gradient descent, Bayesian posterior mean을 전부 정확한 식으로 쓸 수 있어서 Transformer의 forward pass가 어떤 알고리즘과 닮았는지 비교할 수 있습니다.”

3

von Oswald et al.: attention이 GD step을 계산할 수 있다

3.1 먼저 reference GD update

Reference linear model을

$$y(x) = Wx, \quad W \in \mathbb{R}^{d_y \times d_x}$$

라고 하자. Context는

$$D = \{(x_i, y_i)\}_{i=1}^N.$$

Loss는 원 논문과 같이

$$L(W) = \frac{1}{2N} \sum_{i=1}^N \|Wx_i - y_i\|^2.$$

Matrix derivative는

$$\nabla_W \frac{1}{2} \|Wx_i - y_i\|^2 = (Wx_i - y_i)x_i^\top.$$

따라서

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N (Wx_i - y_i)x_i^\top.$$

Learning rate η 의 한 GD step은

$$\Delta W = -\eta \nabla_W L(W) = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)x_i^\top.$$

즉

$$W^+ = W + \Delta W.$$

어떤 query x_j 에 대한 prediction 변화는

$$W^+x_j - Wx_j = \Delta Wx_j = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j).$$

이 식에 이미 attention 모양이 보인다. 각 context point의 residual $(Wx_i - y_i)$ 가 query와의 inner product $x_i^\top x_j$ 로 가중되어 합쳐진다.

3.2 Linear self-attention

von Oswald et al.은 softmax를 제거한 linear self-attention(LSA)을 사용한다. 한 head의 update를 단순화하면

$$e_j^+ = e_j + PVK^\top q_j,$$

여기서

$$V = [W_V e_1, \dots, W_V e_N], \quad K = [W_K e_1, \dots, W_K e_N], \quad q_j = W_Q e_j.$$

이를 합으로 쓰면

$$PVK^\top q_j = P \sum_{i=1}^N (W_V e_i)(W_K e_i)^\top W_Q e_j.$$

즉

$$PVK^\top q_j = P \sum_{i=1}^N v_i (k_i^\top q_j)$$

이다.

3.3 Proposition 1의 핵심 construction을 직접 계산

Token을

$$e_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

로 둔다. 다음 block matrix를 생각하자.

$$W_K = W_Q = \begin{bmatrix} I_{d_x} & 0 \\ 0 & 0 \end{bmatrix}.$$

그러면

$$k_i = W_K e_i = \begin{bmatrix} x_i \\ 0 \end{bmatrix}, \quad q_j = W_Q e_j = \begin{bmatrix} x_j \\ 0 \end{bmatrix}$$

이므로

$$k_i^\top q_j = x_i^\top x_j.$$

Value projection은

$$W_V = \begin{bmatrix} 0 & 0 \\ W & -I_{d_y} \end{bmatrix}$$

로 잡는다. 그러면

$$v_i = W_V e_i = \begin{bmatrix} 0 \\ Wx_i - y_i \end{bmatrix}.$$

마지막으로 output projection의 y block을 η/N 로 두면

$$P = \begin{bmatrix} 0 & 0 \\ 0 & \frac{\eta}{N} I_{d_y} \end{bmatrix}$$

처럼 선택할 수 있다.

따라서 attention update의 y coordinate는

$$\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j).$$

그런데 앞에서

$$\Delta W x_j = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j)$$

였으므로

$$\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j) = -\Delta W x_j.$$

따라서

$$e_j^+ = \begin{bmatrix} x_j \\ y_j - \Delta W x_j \end{bmatrix}$$

가 된다.

원 논문 기준 von Oswald et al.의 Proposition 1은 “학습된 모든 Transformer가 GD를 한다”는 theorem이 아니다. 특정 linear-attention architecture와 tokenization에 대해, 한 attention layer의 data transformation을 한 linear-regression GD step과 정확히 같게 만드는 parameter construction이 존재한다는 결과다. 논문은 그 뒤 실제 meta-training이 이 construction과 매우 가까운 solution을 찾는 경우를 실험적으로 보인다.

3.4 왜 data를 업데이트하는 것이 parameter를 업데이트하는 것과 같은가

처음 보면 attention은 W 를 실제로 변경하지 않는데 어떻게 GD라고 부를 수 있는지 혼란스럽다. 핵심은 query prediction만 놓고 보면 두 표현이 동등하다는 것이다.

GD 뒤 prediction은

$$(W + \Delta W)x_j = Wx_j + \Delta Wx_j.$$

반대로 token의 target-like coordinate를

$$y'_j = y_j - \Delta Wx_j$$

로 바꾸면 residual은

$$Wx_j - y'_j = Wx_j - (y_j - \Delta Wx_j) = (W + \Delta W)x_j - y_j.$$

즉

$$\text{parameter를 } W \rightarrow W + \Delta W \text{로 바꾸는 것} \iff \text{residual 좌표를 적절히 변환하는 것}$$

이라는 dual description을 쓸 수 있다.

3.5 Fast-weight 관점

Linear attention은

$$VK^\top = \sum_{i=1}^N v_i k_i^\top$$

라는 context-dependent matrix를 즉석에서 만든다. 이를

$$S_D := \sum_{i=1}^N v_i k_i^\top$$

라고 하면

$$\text{output} = PS_D q.$$

S_D 는 model parameter에는 저장되어 있지 않고 context에서 매 forward pass마다 새로 만들어 지므로 fast weight처럼 볼 수 있다.

이 관점에서 slow parameter는 P, W_V, W_K, W_Q 이고, fast task-specific object는 S_D 다.

3.6 여러 layer와 curvature correction

하나의 layer가 한 update를 흉내 낼 수 있다면 K 개의 layer는 반복 update와 연결된다. Plain GD는 Hessian이 anisotropic할 때 느릴 수 있다. Linear regression에서

$$H = \nabla^2 L(W) \propto XX^\top \quad \text{또는} \quad X^\top X$$

이고, ideal preconditioned step은 개념적으로

$$w_{t+1} = w_t - \eta M_t \nabla L(w_t)$$

처럼 쓸 수 있다.

von Oswald et al.은 trained Transformer가 plain GD보다 나은 성능을 보일 때, 뒤 layer들이 data-dependent transformation을 이용해 iterative curvature correction에 가까운 계산을 학습할 수 있음을 관찰한다.

주의 여기서 “Transformer = gradient descent”라고 일반화하면 안 된다. 정확한 construction은 linear self-attention, 특정 regression tokenization, 특정 parameter block을 사용한다. Softmax attention과 자연어 LLM에 대해서는 이것이 가능한 메커니즘의 한 예이며, 보편적인 유일한 설명이라는 theorem은 아니다.

발표에서 이렇게 말하기 “핵심 계산은 residual times similarity입니다. GD에서 query prediction의 변화는 $\sum_i (Wx_i - y_i)(x_i^\top x_*)$ 이고, linear attention도 value에 residual을, key-query score에 $x_i^\top x_*$ 를 넣으면 똑같은 합을 만듭니다. 그래서 attention 한 층이 GD 한 step과 정확히 같은 data transformation을 구현할 수 있습니다.”

4

Akyürek et al.: Transformer 안의 암묵적 학습 알고리즘

4.1 질문을 더 강하게 바꾸기

von Oswald의 질문은 “attention이 GD step을 만들 수 있는가?”에 가깝다. Akyürek et al.은 한 단계 더 나아가

Transformer forward pass가 표준 estimator algorithm 자체를 구현할 수 있는가?

를 묻는다. Linear model에서는 정답을 알고 있으므로 construction을 만들고, 실제 trained model의 prediction과 hidden representation을 비교할 수 있다.

4.2 GD implementation theorem

Akyürek et al.은 Transformer decoder layer들이 move, multiply, divide, affine transform 같은 기본 연산을 구현할 수 있음을 construction으로 보인다. 그 building block을 조합하면 linear regression의 GD update를 계산할 수 있다.

Ridge objective의 한 sample contribution을

$$\ell_i(w) = (w^\top x_i - y_i)^2 + \lambda \|w\|^2$$

라고 하면

$$\nabla_w \ell_i(w) = 2x_i(w^\top x_i - y_i) + 2\lambda w.$$

따라서

$$w' = w - 2\alpha [x_i(w^\top x_i - y_i) + \lambda w].$$

원 논문 기준 Akyürek et al.의 Theorem 1은 한 in-context example에 대한 gradient-descent prediction을 constant number of Transformer layers와 $O(d)$ hidden space로 구현할 수 있음을 construction으로 보인다. 이 역시 existence theorem이며, SGD로 학습한 Transformer가 반드시 그 exact circuit을 찾는다는 보장은 아니다.

4.3 Ridge regression을 online update로 쓰기

Ridge solution은

$$w_n = (X_n^\top X_n + \lambda I)^{-1} X_n^\top y_n.$$

이를 매 context example이 들어올 때 다시 inverse 계산하면 비효율적이다. Sherman–Morrison identity를 쓰면 inverse를 순차적으로 갱신할 수 있다.

보충 유도 — 논문 밖의 독자용 전개 — Sherman–Morrison에서 recursive least squares까지

$$A_n = \lambda I + \sum_{i=1}^n x_i x_i^\top, \quad P_n = A_n^{-1}.$$

새 sample x_n 가 들어오면

$$A_n = A_{n-1} + x_n x_n^\top.$$

Sherman–Morrison identity

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u}$$

에 $u = v = x_n$ 을 넣으면

$$P_n = P_{n-1} - \frac{P_{n-1}x_n x_n^\top P_{n-1}}{1 + x_n^\top P_{n-1}x_n}.$$

Gain vector를

$$k_n = \frac{P_{n-1}x_n}{1 + x_n^\top P_{n-1}x_n}$$

라고 하면

$$P_n = P_{n-1} - k_n x_n^\top P_{n-1}.$$

그리고 ridge estimate는 정확히

$$w_n = w_{n-1} + k_n(y_n - x_n^\top w_{n-1})$$

의 recursive least-squares 형태로 갱신할 수 있다.

4.4 위 보충 유도의 마지막 식을 다시 확인

위 boxed recurrence를 직접 확인하자. $b_n = \sum_{i=1}^n x_i y_i$ 라 두면

$$w_n = P_n b_n = P_n(b_{n-1} + x_n y_n).$$

$P_n b_{n-1}$ 에 $P_n = P_{n-1} - k_n x_n^\top P_{n-1}$ 를 넣으면

$$P_n b_{n-1} = w_{n-1} - k_n x_n^\top w_{n-1}.$$

또 Sherman–Morrison 식으로부터

$$P_n x_n = k_n.$$

따라서

$$w_n = w_{n-1} - k_n x_n^\top w_{n-1} + k_n y_n,$$

즉

$$w_n = w_{n-1} + k_n(y_n - x_n^\top w_{n-1}).$$

이 식은 “현재 residual”에 adaptive gain을 곱해 update한다는 점에서 GD와 닮았지만, k_n 안에 inverse covariance P_{n-1} 가 들어 있다는 점이 다르다.

4.5 왜 depth와 noise에 따라 다른 알고리즘처럼 보일 수 있는가

Akyürek et al.의 empirical result에서 trained ICL predictor는 조건에 따라 GD, ridge, least squares와 서로 다른 정도로 정렬된다. 이것은 하나의 모델이 항상 하나의 closed-form algorithm

만 실행한다는 단순한 그림보다, training distribution과 architecture budget에 맞는 estimator를 학습한다는 쪽과 더 잘 맞는다.

핵심 같은 input-output behavior를 여러 algorithm이 만들 수 있다. 특히 noiseless full-rank linear regression에서는 충분히 수렴한 GD, OLS, 약한 regularization의 ridge가 거의 같은 predictor를 낸다. 내부 mechanism을 주장하려면 test error 만이 아니라 hidden-state probe, intervention, parameter construction 같은 추가 증거가 필요하다.

발표에서 이렇게 말하기 “Akyürek 논문의 중요한 확장은 Transformer가 단순히 GD 한 step만 훔쳐 낼 수 있다는 것을 넘어서, ridge regression 같은 estimator를 순차적 matrix update로 구현할 수 있다는 점입니다. 그래서 ICL을 hidden state 안에서 돌아가는 작은 학습 알고리즘으로 볼 수 있습니다.”

5

Bayesian linear regression: optimization과 inference가 만나는 곳

5.1 Gaussian prior와 likelihood

Task-specific weight에 prior를 둔다.

$$w \sim \mathcal{N}(0, \tau^2 I).$$

Context label은

$$y = Xw + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_n).$$

따라서

$$p(y | X, w) \propto \exp\left(-\frac{1}{2\sigma^2} \|y - Xw\|^2\right),$$

$$p(w) \propto \exp\left(-\frac{1}{2\tau^2} \|w\|^2\right).$$

Bayes rule로

$$p(w | X, y) \propto p(y | X, w)p(w).$$

Log posterior의 w 에 의존하는 부분은

$$-\log p(w | X, y) = \frac{1}{2\sigma^2} \|y - Xw\|^2 + \frac{1}{2\tau^2} \|w\|^2 + C.$$

따라서 MAP estimator는

$$\arg \min_w \left[\|y - Xw\|^2 + \frac{\sigma^2}{\tau^2} \|w\|^2 \right].$$

즉 ridge parameter를

$$\lambda = \frac{\sigma^2}{\tau^2}$$

로 두면

$$w_{\text{MAP}} = \hat{w}_\lambda.$$

5.2 Posterior를 완전히 유도

Quadratic term을 모으면

$$-2\log p(w | X, y) = \frac{1}{\sigma^2} (y - Xw)^\top (y - Xw) + \frac{1}{\tau^2} w^\top w + C.$$

첫 항을 펼치면

$$(y - Xw)^\top (y - Xw) = y^\top y - 2w^\top X^\top y + w^\top X^\top X w.$$

따라서 w 관련 항은

$$w^\top \left(\frac{1}{\sigma^2} X^\top X + \frac{1}{\tau^2} I \right) w - 2w^\top \frac{1}{\sigma^2} X^\top y.$$

Posterior precision을

$$\Lambda = \frac{1}{\sigma^2} X^\top X + \frac{1}{\tau^2} I$$

라고 하면 covariance는

$$\Sigma = \Lambda^{-1}.$$

Complete the square로 posterior mean은

$$\mu = \Sigma \frac{1}{\sigma^2} X^\top y.$$

따라서

$$\boxed{p(w \mid X, y) = \mathcal{N}(\mu, \Sigma)}.$$

Σ 를 다시 쓰면

$$\Sigma = \sigma^2 \left(X^\top X + \frac{\sigma^2}{\tau^2} I \right)^{-1},$$

그래서

$$\mu = \left(X^\top X + \frac{\sigma^2}{\tau^2} I \right)^{-1} X^\top y.$$

즉 Gaussian model에서는

$$\boxed{\text{posterior mean} = \text{extMAP} = \text{extridgeestimator}}$$

이다.

5.3 Posterior predictive

새 query x_* 에 대해

$$y_* = x_*^\top w + \varepsilon_*, \quad \varepsilon_* \sim \mathcal{N}(0, \sigma^2).$$

Posterior predictive mean은

$$\mathbb{E}[y_* \mid x_*, X, y] = x_*^\top \mathbb{E}[w \mid X, y] = x_*^\top \mu.$$

분산은 law of total variance로

$$\text{Var}(y_* \mid x_*, X, y) = \mathbb{E}[\text{Var}(y_* \mid w, x_*) \mid D] + \text{Var}(\mathbb{E}[y_* \mid w, x_*] \mid D).$$

첫 항은 σ^2 , 둘째 항은 $x_*^\top \Sigma x_*$ 이므로

$$\boxed{y_* \mid x_*, D \sim \mathcal{N}\left(x_*^\top \mu, \sigma^2 + x_*^\top \Sigma x_*\right)}.$$

5.4 GD, ridge, Bayes가 언제 같은가

이제 앞 장들의 관계가 명확해진다.

- Gaussian prior + Gaussian noise에서 Bayes posterior mean은 ridge와 정확히 같다.
- $\tau^2 \rightarrow \infty$ 이면 prior가 평평해져 $\lambda = \sigma^2/\tau^2 \rightarrow 0$ 이고, full rank에서 OLS로 간다.
- Stable GD를 충분히 오래 돌리면 unregularized square loss에서는 OLS로 간다.

따라서 특정 regime에서는

$$\text{many-step GD} \approx \text{OLS} \approx \text{weak-prior Bayesian predictor}$$

가 될 수 있다.

주의 이 동등성은 Gaussian linear model이라는 매우 특수한 경우다. 일반 nonlinear model, non-Gaussian prior, misspecified likelihood에서는 posterior mean, MAP, gradient-descent iterate가 서로 다르다. “ICL은 GD인가 Bayes인가”를 하나의 yes/no 질문으로 만들면 이 구분을 놓치게 된다.

발표에서 이렇게 말하기 “Linear-Gaussian setting에서는 GD와 Bayes 관점이 실제로 만납니다. Gaussian prior의 posterior mean은 ridge이고, prior를 약하게 하면 OLS가 되며, square-loss GD를 충분히 돌려도 OLS로 갑니다. 그래서 같은 prediction behavior만 보고 ‘GD다’ 또는 ‘Bayes다’라고 구분하기는 어렵습니다.”

6

Xie et al.: ICL을 latent-concept Bayesian inference로 보기

6.1 Pretraining distribution의 latent concept

Xie et al.은 자연어 pretraining document가 문서 전체에 걸쳐 공유되는 latent concept를 가진다고 본다. Concept을 $\theta \in \Theta$ 라 하면 pretraining document $o_{1:T}$ 의 distribution은

$$p_{\text{pre}}(o_{1:T}) = \int_{\Theta} p(o_{1:T} | \theta) p(\theta) d\theta.$$

논문에서는 $p(o_{1:T} | \theta)$ 를 HMM으로 구체화한다. Concept θ 가 hidden-state transition 구조를 결정하고, 관측 token은 hidden state에서 생성된다.

LM이 p_{pre} 를 정확히 학습했다고 이상화하면 next-token predictor는

$$p_{\text{pre}}(o_{t+1} | o_{1:t})$$

이고, latent concept을 전개하면

$$p(o_{t+1} | o_{1:t}) = \int p(o_{t+1} | o_{1:t}, \theta) p(\theta | o_{1:t}) d\theta.$$

즉 next-token prediction 자체가 posterior averaging을 포함한다.

6.2 Prompt가 왜 posterior를 바꾸는가

Prompt가

$$O_1, o^{\text{delim}}, O_2, o^{\text{delim}}, \dots, O_n$$

처럼 여러 example을 delimiter로 연결한 형태라고 하자. 각 example이 같은 concept θ^* 에서 왔다면 posterior는 개념적으로

$$p(\theta | O_{1:n}) \propto p(\theta) p(O_{1:n} | \theta)$$

로 갱신된다.

Context가 늘어날수록 θ^* 를 지지하는 likelihood evidence가 누적되면

$$p(\theta^* | O_{1:n}) \rightarrow 1$$

이 될 수 있다. 그러면 posterior predictive는

$$p(o_* | O_{1:n}) \approx p(o_* | O_{1:n}, \theta^*)$$

가 되어 task-specific prediction이 나온다.

6.3 Prompt와 pretraining distribution의 mismatch

실제 prompt는 서로 독립적인 example들을 delimiter로 붙인 인공적인 sequence일 수 있다. 반면 pretraining document는 하나의 HMM trajectory다. 그러므로

$$p_{\text{prompt}} \neq p_{\text{pre}}$$

일 수 있다.

그런데 LM은 p_{pre} 를 학습했다. 그럼에도 ICL이 되려면 prompt가 pretraining model 아래에서 완전히 엉뚱한 sequence로 해석되지 않고, 각 example 내부가 주는 concept evidence가 example 사이의 비정상적인 transition penalty보다 충분히 강해야 한다.

원 논문 기준 Xie et al.의 main theorem은 delimiter state와 regularity에 대한 가정들, 그리고 concept signal이 cross-example mismatch를 이기는 조건 아래에서, example 수 $n \rightarrow \infty$ 일 때 pretraining distribution의 conditional predictor가 prompt task의 optimal 0-1 risk에 수렴함을 보인다. 즉 theorem의 핵심은 “prompt가 pretraining distribution과 정확히 같아야 한다”가 아니라, 특정 HMM mixture에서는 mismatch가 있어도 posterior concept selection이 성공할 수 있다는 점이다.

6.4 보충: 단순 IID mixture에서는 posterior concentration이 왜 일어나는가

Xie 논문의 HMM proof 전체를 재현하기 전에 더 단순한 IID latent-concept model을 보면 핵심이 선명하다. Concept 후보가 θ^* 와 θ 두 개라고 하자. Example Z_i 들이 실제로 P_{θ^*} 에서 IID로 생성되면 posterior odds는

$$\frac{p(\theta^* | Z_{1:n})}{p(\theta | Z_{1:n})} = \frac{p(\theta^*)}{p(\theta)} \prod_{i=1}^n \frac{p(Z_i | \theta^*)}{p(Z_i | \theta)}.$$

Log를 취하면

$$\log \frac{p(\theta^* | Z_{1:n})}{p(\theta | Z_{1:n})} = \log \frac{p(\theta^*)}{p(\theta)} + \sum_{i=1}^n \log \frac{p(Z_i | \theta^*)}{p(Z_i | \theta)}.$$

n 으로 나누면

$$\frac{1}{n} \log \frac{p(\theta^* | Z_{1:n})}{p(\theta | Z_{1:n})} = \frac{1}{n} \log \frac{p(\theta^*)}{p(\theta)} + \frac{1}{n} \sum_{i=1}^n \log \frac{p(Z_i | \theta^*)}{p(Z_i | \theta)}.$$

Law of large numbers로 두 번째 항은

$$\mathbb{E}_{Z \sim P_{\theta^*}} \left[\log \frac{p(Z | \theta^*)}{p(Z | \theta)} \right] = D_{\text{KL}}(P_{\theta^*} \| P_{\theta})$$

로 간다. Identifiability가 있어

$$D_{\text{KL}}(P_{\theta^*} \| P_{\theta}) > 0$$

이면 posterior odds는 exponential하게 θ^* 쪽으로 쏠린다.

보충 유도 — 논문 밖의 독자용 전개 Xie et al.의 HMM setting에서는 example 내부 evidence 외에 delimiter와 example 사이 transition이 추가된다. 따라서 위 IID derivation의 “양의 KL gap”에 대응하는 signal이 cross-example mismatch penalty를 충분히 이겨야 한다. 이 보충 유도는 논문의 exact theorem statement를 대체하는 것이 아니라, 왜 theorem에 signal-versus-mismatch 조건이 필요한지 설명하기 위한 단순화다.

6.5 Bayesian view에서 demonstration order가 중요할 수 있는 이유

IID Bayes model에서는 product likelihood가 순서에 무관하지만 HMM은

$$p(o_{1:T} \mid \theta)$$

가 transition structure를 포함하므로 순서가 likelihood를 바꿀 수 있다. 따라서 example order sensitivity는 이 framework와 모순되지 않는다. Xie et al.의 GINC 실험에서도 model scaling, order sensitivity, zero-shot이 few-shot보다 나올 수 있는 경우 같은 현상이 관찰된다.

발표에서 이렇게 말하기 “Xie 논문의 핵심은 prompt에서 새 알고리즘을 발명한다기보다, pretraining 중 배운 latent concept 가운데 지금 prompt를 설명하는 concept의 posterior를 올린다는 것입니다. HMM theorem은 prompt가 pretraining 문서와 distribution이 달라도 example 내부의 concept signal이 boundary mismatch보다 강하면 이 inference가 성공할 수 있음을 보입니다.”

7

Meta-learning으로 세 관점을 하나의 그림에 놓기

7.1 Outer loop와 inner loop

Meta-learning의 표준 표기를 쓰면 task τ 마다 support set D_τ 가 있고, inner learner 가 task-specific predictor parameter 를 만든다.

$$\phi_\tau = A_\Theta(D_\tau).$$

Query prediction 은

$$\hat{y}_* = f_{\phi_\tau}(x_*).$$

Outer objective 는

$$\min_{\Theta} \mathbb{E}_{\tau} [L_{\tau}^{\text{query}}(A_{\Theta}(D_{\tau}))].$$

ICL Transformer에서는 A_{Θ} 가 별도 optimizer 함수로 호출되지 않는다. 대신

$$F_{\Theta}(D_{\tau}, x_*)$$

라는 하나의 forward pass가 inner adaptation과 query prediction을 동시에 수행한다.

7.2 MAML과 ICL의 차이

MAML류에서는 inner loop가 명시적으로

$$\phi_\tau = \Theta - \alpha \nabla_{\Theta} L_{\tau}^{\text{support}}(\Theta)$$

처럼 model parameter의 복사본을 업데이트한다.

ICL에서는

$$\Theta \text{ fixed, } h_{1:L} = \text{TransformerForward}_{\Theta}(D_{\tau}, x_*),$$

이고 hidden activation h 가 task-specific state를 담는다.

따라서 두 방법은 모두 “outer loop가 inner adaptation rule을 학습한다”는 메타-learning 관점으로 묶을 수 있지만, inner state가 저장되는 위치가 다르다.

	Gradient-based meta-learning	In-context learning
slow state	initialization / optimizer parameter	Transformer weights Θ
fast state	adapted parameter ϕ_τ	activations, attention-derived statistics
inner update	explicit gradient step	forward computation
new-task data	support set	prompt/context
query evaluation	adapted model forward	same Transformer forward

7.3 Amortized inference 관점

Bayesian inference를 매 task마다 정확히 계산하려면

$$p(\theta | D) \propto p(D | \theta)p(\theta)$$

를 새로 계산해야 한다. 하지만 task distribution을 반복해서 본다면 neural network가

$$D \mapsto \text{posterior summary}$$

라는 inference map 자체를 학습할 수 있다. 이를 amortized inference라고 부른다.

ICL Transformer는

$$D_n, x_* \mapsto \hat{y}_*$$

를 한 번의 forward pass로 계산하므로, Bayesian view에서는 posterior-predictive computation을 amortize한 모델로 해석할 수 있다. Optimization view에서는 학습 알고리즘 자체를 amortize했다고 해석할 수 있다.

7.4 세 관점이 동시에 참일 수 있는 예

Gaussian linear regression을 다시 보자.

1. Outer training은 여러 w_τ task에 대해 prediction error를 줄인다.
2. Forward pass가 GD를 여러 step 근사한다.
3. 충분히 깊으면 OLS에 가까워진다.
4. Gaussian prior/noise에서는 target Bayes posterior mean이 ridge다.
5. weak prior에서 ridge는 OLS에 가까워진다.

따라서 같은 predictor가

meta-learned optimizer, least-squares algorithm, Bayesian posterior estimator

세 언어로 모두 설명될 수 있다.

핵심 좋은 이론 질문은 “ICL의 진짜 정체는 GD인가 Bayes인가?”가 아니라, 어떤 **data model**과 **architecture**에서 어떤 **observable prediction** 또는 **internal computation**이 어느 **algorithm**과 동등한가?이다.

발표에서 이렇게 말하기 “Meta-learning은 세 관점을 묶는 상위 그림입니다. Training은 여러 task에 걸쳐 slow weights에 learning rule을 새기고, inference에서는 prompt가 fast state를 만듭니다. 그 fast computation이 어떤 문제에서는 GD처럼, 어떤 문제에서는 ridge나 Bayesian posterior처럼 보일 수 있습니다.”

8

무엇이 theorem이고 무엇이 해석인가

8.1 네 논문의 claim을 같은 강도로 읽으면 안 된다

논문	강한 수학적 진술	조심해서 읽을 부분
Garg et al.	meta-trained Transformer가 여러 simple function class에서 ICL을 수행함을 체계적으로 실험	mechanism 자체를 unique하게 식별하지 않음
von Oswald et al.	1-head linear attention이 linear-regression GD step과 동일한 transformation을 구현하는 explicit construction	standard softmax LLM 전체가 항상 GD를 한다는 theorem이 아님
Akyürek et al.	Transformer가 GD와 ridge-related computation을 구현할 수 있는 construction과 complexity upper bound	실제 trained model의 exact internal algorithm은 behavioral/probing evidence와 함께 해석
Xie et al.	mixture-of-HMM pretraining model에서 조건 아래 implicit Bayesian ICL의 asymptotic guarantee	실제 web-scale language distribution이 그 HMM model과 동일하다는 주장은 아님

8.2 Can implement와 does implement

Existence theorem은

$$\exists \Theta : F_{\Theta} = \mathcal{A}$$

를 보인다. 하지만 training dynamics가 그 parameter를 찾는다는 것은

$$\Theta_{\text{train}} \approx \Theta_{\mathcal{A}}$$

라는 별도의 optimization claim이다.

또 prediction이 같다는 것은

$$F_{\Theta_{\text{train}}}(D, x) \approx \mathcal{A}(D, x)$$

라는 behavioral claim이고, 내부 state가 실제 algorithm variable을 encode한다는 것은 더 강한 mechanistic claim이다.

따라서 증거의 강도를

$$\text{expressibility} < \text{behavioral match} < \text{mechanistic identification}$$

처럼 구분해서 보는 것이 좋다. 이 부등호는 논리적 implication을 뜻한다기보다 증거 요구량의 직관적 순서를 나타낸다.

8.3 Linear attention과 softmax attention

von Oswald의 exact proof는

$$\text{Attn}(q, K, V) = V K^{\top} q$$

형태의 linear attention을 사용한다. Standard attention은

$$\text{Attn}(q, K, V) = V \text{softmax}(K^\top q)$$

이므로 weight가 positive하고 normalization을 거친다. 두 연산은 일반적으로 같지 않다.

따라서 linear-attention construction을 GPT류 softmax Transformer에 그대로 대입하는 것은 theorem의 범위를 넘는다. 다만 multi-layer nonlinear Transformer가 비슷한 effective computation을 학습할 수 있는지는 empirical/mechanistic question으로 남는다.

8.4 Toy regression에서 자연어 ICL로 갈 때 추가되는 것

자연어에서는 task가 명시적인 w 하나로 표현되지 않을 수 있다. 또 context example의 의미를 해석하고 input/output field를 분리하는 representation learning이 먼저 필요하다. 개념적으로는

$$\text{raw token} \xrightarrow{\text{representation}} \text{task-relevant variable} \xrightarrow{\text{inner algorithm}} \text{prediction}$$

으로 분해할 수 있다.

von Oswald et al.도 MLP가 있는 Transformer에서 learned representation 위의 linear model을 in-context로 학습하는 관점을 논의한다. 하지만 실제 LLM에서는 여러 mechanism이 중첩될 가능성이 높다.

8.5 분포 밖 일반화와 task recognition

ICL이 정말 “새로운 task를 학습”한 것인지, 아니면 pretraining에서 이미 본 task family를 “식별”한 것인지도 구분해야 한다. Bayesian latent-concept view는 후자와 자연스럽게 연결된다. 반면 Garg et al.의 synthetic function-class setting은 unseen function parameter에 대한 within-family generalization을 명확히 측정한다.

이 둘은

$$\text{new parameter within known family} \neq \text{new function family outside training support}$$

이다.

발표에서 이렇게 말하기 “이번 주 논문을 읽을 때 제일 중요한 경계는 ‘can implement’와 ‘does implement’입니다. Construction은 Transformer에 그런 알고리즘을 넣을 수 있음을 보일 뿐이고, 실제 학습된 모델이 그 알고리즘을 쓴다는 주장은 prediction matching이나 internal representation evidence가 추가로 필요합니다.”

9

발표 준비용 압축 정리

9.1 칠판에 반드시 남길 열두 개 식

1. ICL predictor

$$\hat{y}_* = F_{\Theta}(D_n, x_*), \quad \Theta \text{ fixed at inference.}$$

2. Meta objective

$$\min_{\Theta} \mathbb{E}_{\tau, D, x_*} [\ell(F_{\Theta}(D, x_*), y_*)].$$

3. Linear task

$$y = Xw + \varepsilon.$$

4. OLS

$$\hat{w}_{\text{OLS}} = (X^{\top} X)^{-1} X^{\top} y.$$

5. Ridge

$$\hat{w}_{\lambda} = (X^{\top} X + \lambda I)^{-1} X^{\top} y.$$

6. GD update

$$\Delta W = -\frac{\eta}{N} \sum_i (W x_i - y_i) x_i^{\top}.$$

7. Query prediction change

$$\Delta W x_* = -\frac{\eta}{N} \sum_i (W x_i - y_i) (x_i^{\top} x_*).$$

8. Linear attention

$$P V K^{\top} q = P \sum_i v_i (k_i^{\top} q).$$

9. Recursive ridge gain

$$k_n = \frac{P_{n-1} x_n}{1 + x_n^{\top} P_{n-1} x_n}.$$

10. Recursive estimator update

$$w_n = w_{n-1} + k_n (y_n - x_n^{\top} w_{n-1}).$$

11. Bayesian posterior

$$\Sigma = \left(\sigma^{-2} X^{\top} X + \tau^{-2} I \right)^{-1}, \quad \mu = \Sigma \sigma^{-2} X^{\top} y.$$

12. Latent-concept pretraining model

$$p_{\text{pre}}(o_{1:T}) = \int p(o_{1:T} \mid \theta) p(\theta) d\theta.$$

9.2 발표 순서 추천

1. ICL은 inference-time parameter update가 없다는 점부터 명확히 한다.
2. Linear regression을 공통 testbed로 소개하고 OLS/GD/ridge를 한 장에서 맞춘다.
3. von Oswald의 $\sum_i \text{residual}_i \times \text{similarity}_{i,*}$ 식을 중심으로 attention-GD equivalence를 유도한다.
4. “can implement”와 “learned model actually matches”를 분리한다.
5. Akyürek의 ridge / recursive estimator 관점으로 algorithm class를 확장한다.
6. Gaussian Bayesian linear regression을 직접 유도해 ridge와 posterior mean의 동등성을 보인다.
7. Xie의 latent-concept HMM picture로 자연어 pretraining과 연결한다.
8. 마지막으로 GD vs Bayes가 반드시 양자택일이 아니라는 점을 정리한다.

9.3 자주 나오는 질문

Q1. Parameter update가 없는데 왜 learning이라고 하나?

Task-specific rule이 context에 따라 변하기 때문이다. 그 state가 parameter가 아니라 activation 또는 context-dependent matrix에 저장될 수 있다.

Q2. von Oswald가 Transformer가 실제로 backpropagation을 한다고 증명했나?

아니다. Linear self-attention의 forward computation이 linear regression의 GD-induced transformation과 같아지는 explicit parameter construction을 보인 것이다.

Q3. Attention score가 softmax인데 어떻게 signed gradient를 만들 수 있나?

Exact Proposition 1은 softmax를 제거한 linear self-attention을 사용한다. Standard softmax LLM에 그대로 적용되는 exact identity가 아니다.

Q4. ICL이 GD라면 왜 ridge나 Bayes와도 비슷한가?

Linear-Gaussian problem에서는 이 estimator들이 서로 가까워지거나 정확히 일치하는 regime이 있다. Output behavior만으로 내부 algorithm을 유일하게 정할 수 없다.

Q5. Bayesian view에서는 새로운 task를 진짜 배우는 게 아니라 prior에서 찾기만 하는 것 아닌가?

Xie et al.의 formal model에서는 latent concept inference가 핵심이다. 반면 synthetic function-class 연구에서는 unseen parameter/function에 대한 adaptation을 측정한다. “새롭다”의 의미가 setting마다 다르다.

Q6. Context가 길어지면 항상 좋아지나?

아니다. Bayesian model에서도 misspecification이나 misleading examples가 있으면 posterior가 나빠질 수 있고, finite Transformer에서는 context length, attention, representation bottleneck이 있다.

Q7. OLS와 GD가 같은 prediction이면 mechanism을 어떻게 구분하나?

Depth dependence, intermediate representation, intervention, out-of-distribution behavior, algorithm-specific state probe 등을 봐야 한다.

9.4 수학적 실수 체크리스트

- ICL inference 중 Θ 가 업데이트된다고 쓰지 않았는가?

- $X \in \mathbb{R}^{n \times d}$ convention에서 $X^\top X$ 와 $XX^\top$ 을 뒤집지 않았는가?
- OLS inverse가 존재하려면 rank condition이 필요하다는 점을 잊지 않았는가?
- von Oswald의 exact construction이 linear self-attention이라는 점을 생략하지 않았는가?
- $\Delta W = -\eta \nabla L$ 의 부호와 token transformation $y_j - \Delta W x_j$ 의 부호를 혼동하지 않았는가?
- Ridge의 λ convention과 Bayesian σ^2/τ^2 scaling을 맞췄는가?
- Posterior mean과 MAP이 일반적으로 항상 같은 것이 아니라 Gaussian quadratic case에서 같다는 점을 명시했는가?
- Xie의 HMM theorem을 IID Bayes derivation과 동일한 theorem으로 서술하지 않았는가?
- can implement, behaviorally matches, mechanistically implements를 구분했는가?
- synthetic regression 결과를 실제 자연어 LLM 전체의 일반 법칙으로 확대하지 않았는가?

9.5 30초 결론

In-context learning은 고정된 Transformer weight가 context를 이용해 task-specific predictor를 즉석에서 구성하는 현상이다. Linear regression에서는 이 계산을 정확히 분석할 수 있다. von Oswald et al.은 linear self-attention 한 층이 residual과 input similarity를 결합해 한 GD step과 동일한 transformation을 만들 수 있음을 보였고, Akyürek et al.은 Transformer가 GD와 ridge-regression 관련 계산을 explicit construction으로 구현할 수 있음을 보였다. 반면 Xie et al.은 pretraining document의 latent concept을 추론하는 Bayesian posterior 관점에서 ICL이 왜 나타날 수 있는지 설명한다. Gaussian linear model에서는 ridge가 posterior mean과 정확히 같고 충분히 수렴한 GD가 OLS로 가므로, optimization과 Bayesian 설명은 특정 regime에서 같은 predictor를 기술할 수도 있다.

발표에서 이렇게 말하기 “한 문장으로 정리하면, ICL은 weights를 바꾸는 대신 forward pass 안에 작은 학습 알고리즘을 실행하는 것으로 볼 수 있습니다. 그 알고리즘은 linear regression에서는 GD나 ridge로 구체화할 수 있고, probabilistic 관점에서는 latent task에 대한 posterior inference로 읽을 수 있습니다. 중요한 것은 어느 관점이 '진짜'냐보다 각 theorem이 어떤 setting에서 어떤 동등성을 보였는지입니다.”

참고문헌 및 원문에서 확인할 위치

Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, Max Vladymyrov. “Transformers Learn In-Context by Gradient Descent.” ICML 2023.

- Sec. 2, Eq. (2)–(4): linear regression loss와 GD-induced data transformation.
- Proposition 1: one-head linear self-attention이 한 GD step과 동일한 transformation을 구현하는 construction.
- Sec. 3: trained LSA/Transformer와 GD predictor의 empirical alignment.
- 후반부: iterative curvature correction, learned representation 위의 linear in-context learning.

<https://proceedings.mlr.press/v202/von-oswald23a.html>

Sang Michael Xie, Aditi Raghunathan, Percy Liang, Tengyu Ma. “An Explanation of In-context Learning as Implicit Bayesian Inference.” ICLR 2022.

- Sec. 2: latent concept θ 와 mixture-of-HMM pretraining distribution.
- Assumptions: delimiter hidden states와 transition regularity.
- Theorem 1: paper의 signal/mismatch condition 아래 example 수가 증가할 때 in-context predictor가 optimal 0–1 risk에 수렴.
- GINC experiments: scaling, example-order sensitivity, zero-shot/few-shot phenomena.

<https://arxiv.org/abs/2111.02080>

Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, Denny Zhou. “What Learning Algorithm Is In-Context Learning? Investigations with Linear Models.” ICLR 2023.

- Sec. 3.1: move/multiply/divide/affine primitives의 Transformer construction.
- Theorem 1: constant layers와 $O(d)$ hidden space로 single GD-step prediction을 구현.
- Ridge/least-squares construction: matrix inverse update와 estimator computation.
- Experiments: depth와 noise에 따라 GD, ridge, least-squares predictor와의 behavioral alignment.

<https://arxiv.org/abs/2211.15661>

Shivam Garg, Dimitris Tsipras, Percy Liang, Gregory Valiant. “What Can Transformers Learn In-Context? A Case Study of Simple Function Classes.” NeurIPS 2022.

- Linear function family를 이용한 controlled ICL setup.
- OLS baseline과 in-context error 비교.
- Sparse linear functions, two-layer neural networks, decision trees로의 확장.
- Training/prompt/query distribution shift에 대한 실험.

<https://arxiv.org/abs/2208.01066>

주의— 원문과 이 노트의 경계 Gaussian Bayesian linear regression의 posterior 유도, GD의 closed-form iterate, recursive least-squares 식, IID latent-mixture에서 KL divergence를 이용한 posterior concentration 유도는 원 논문의 핵심 아이디어를 연결하기 위해 추가한 보충 전개다. 특히 IID posterior-concentration 계산은 Xie et al.의 mixture-of-HMM Theorem 1의 proof가 아니며, 그 theorem의 delimiter/transition assumptions를 단순화해서 직관을 제공한 것이다. 또한 von Oswald et al.의 exact Proposition 1은 linear self-attention에 대한 construction이므로 일반 softmax LLM에 대한 보편적 mechanistic theorem으로 확대하지 않았다.